

Implementing and Evaluating KADABRA and BRAVA-GNN in Julia

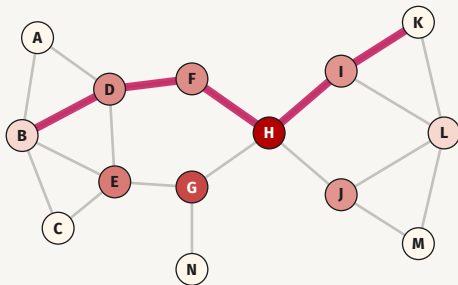
Lab Computational Analytics — Open-Source Graph Software Engineering

Martin Schlaier

Universität Bonn

Supervisors: Dr. Aurora Rossi, Prof. Dr. Petra Mutzel,
Laura Bülte

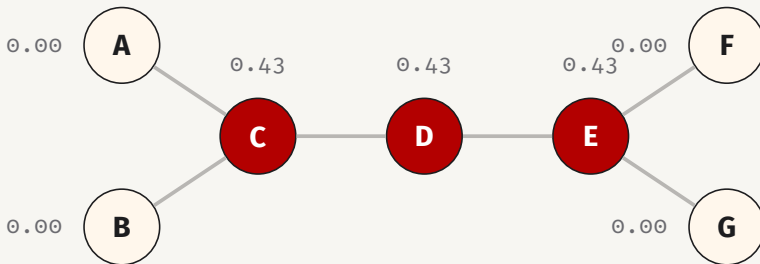
September 16, 2026



Betweenness Centrality

$$bc(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Betweenness Centrality: Example



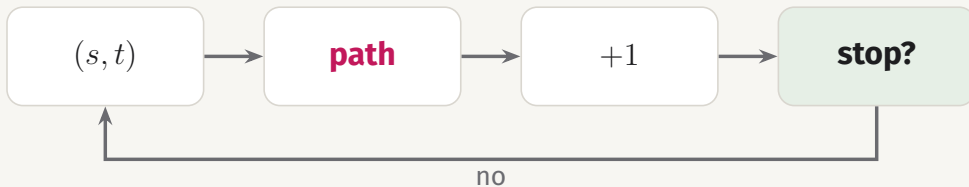
Exact Computation Does Not Scale

$$O(nm)$$

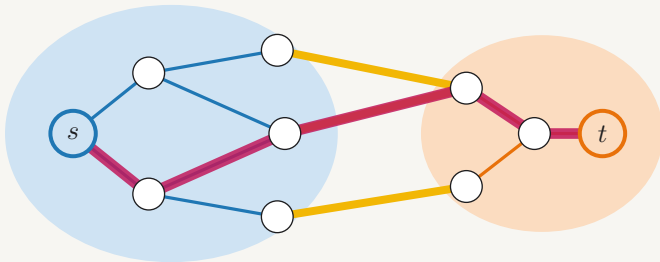
169 min

exact betweenness, email-EuAll

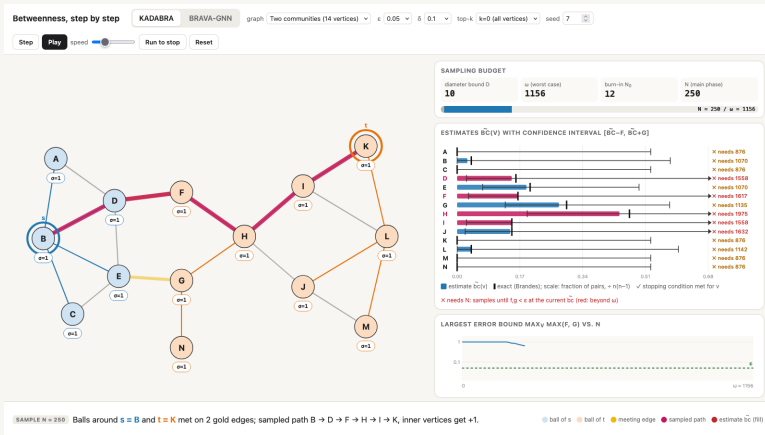
KADABRA: Sample Shortest Paths



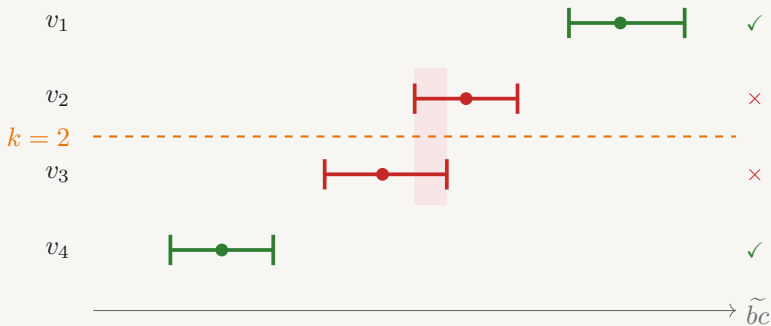
Sampling a Path: Balanced Bidirectional BFS



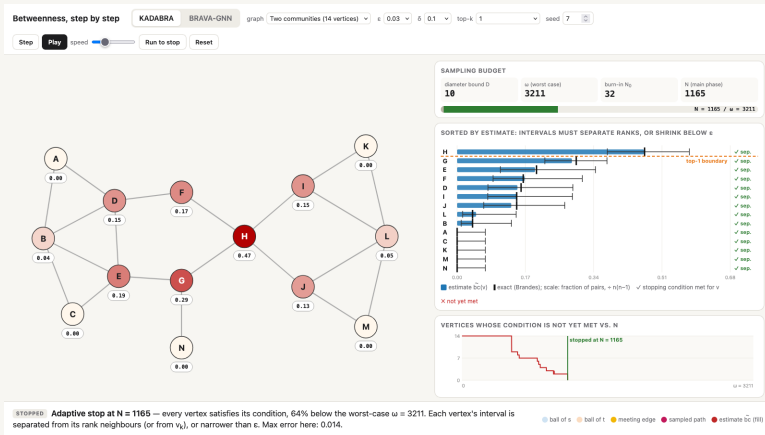
Live Demo: Sampling Paths


[Open live demo](#)

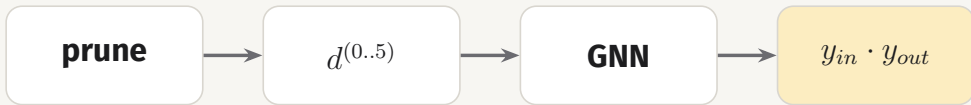
When to Stop: Top- k



Live Demo: Top- k Stops Early

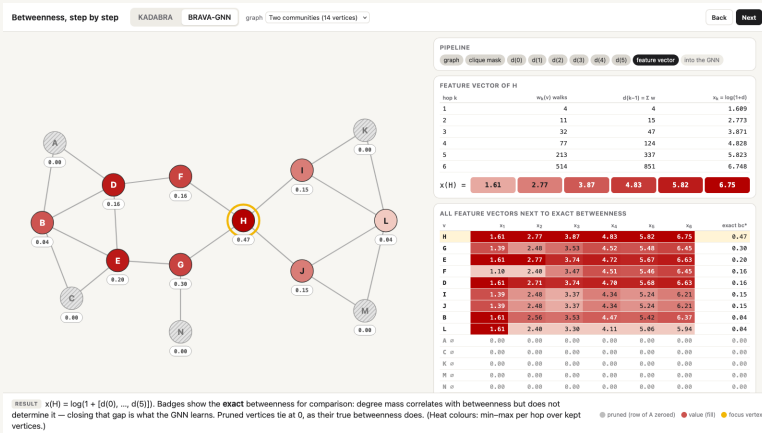

[Open live demo](#)

BRAVA-GNN: Learn the Ranking



≈ 1300 parameters

Live Demo: One Feature Vector


[Open live demo](#)

What We Did



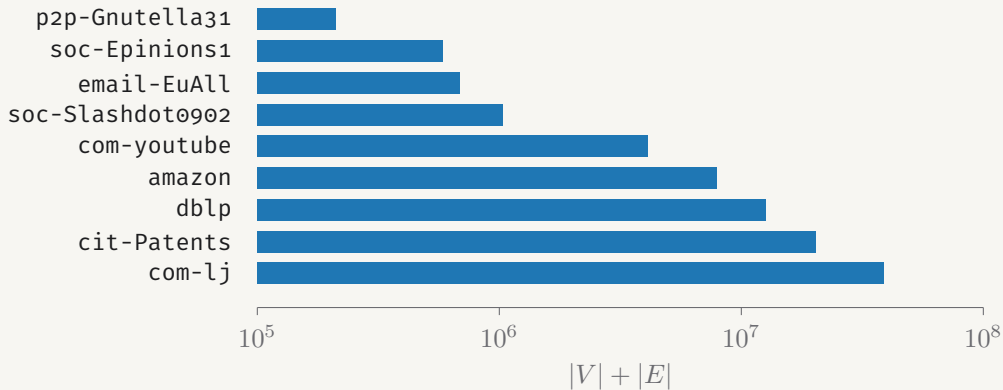
Research Questions

RQ1 Julia vs. C++

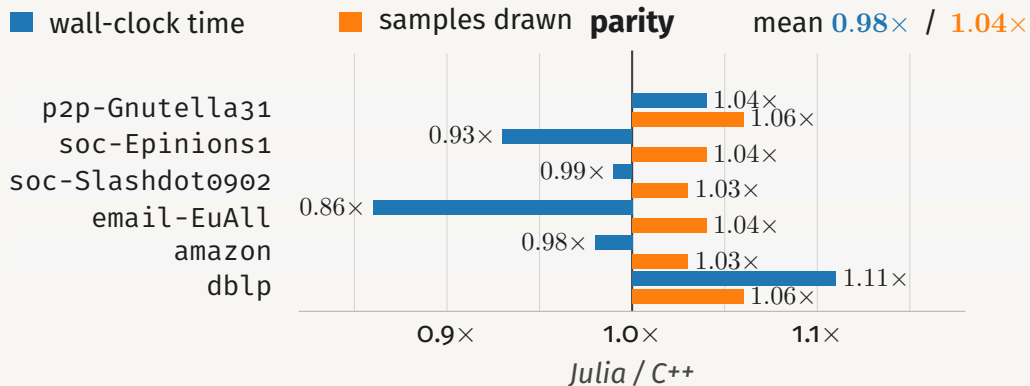
RQ2 BRAVA-GNN vs. KADABRA

RQ3 Does top- k save work?

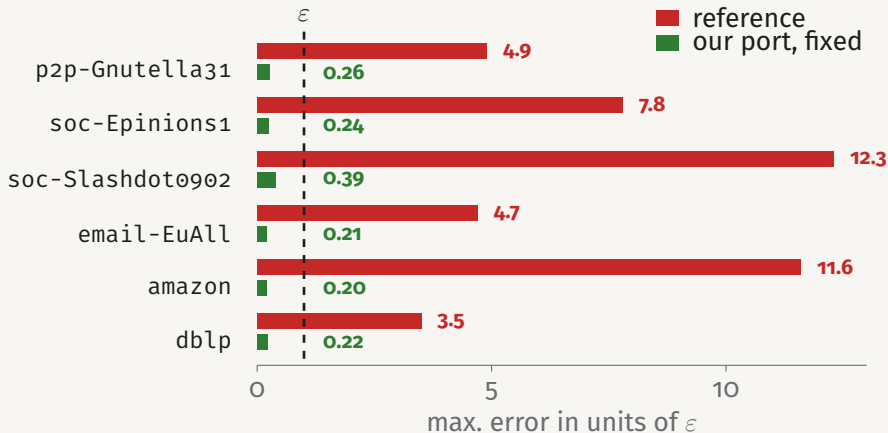
Setup



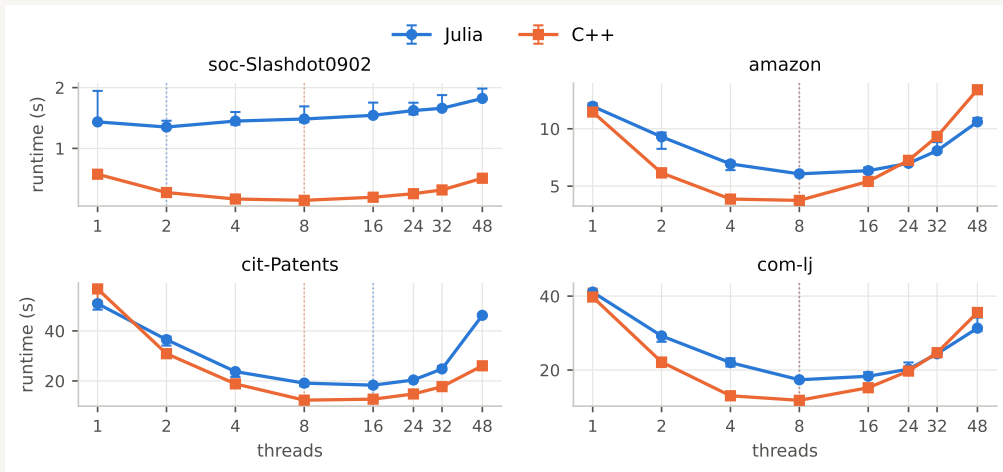
Same Samples, Same Time



A Bug We Copied: Scores Too Low



Thread Scaling



Answer to RQ1

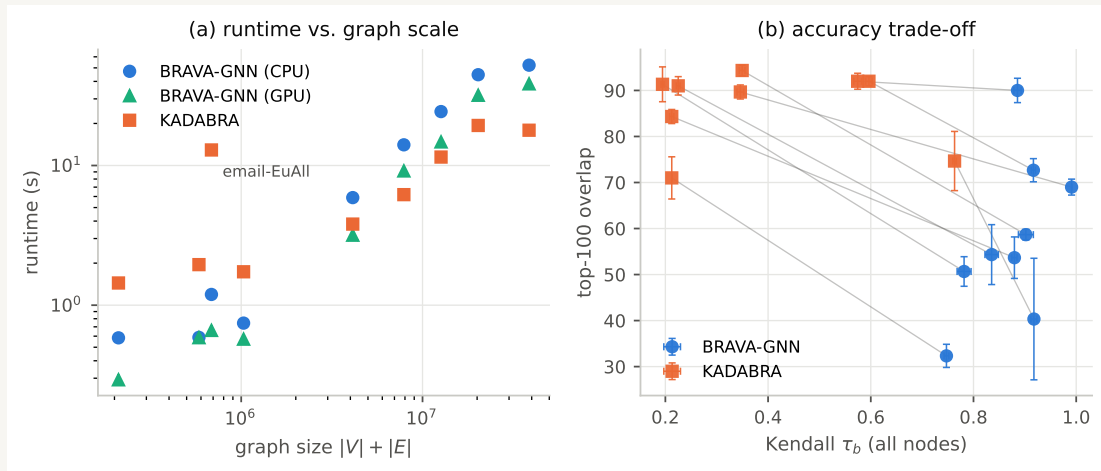
Julia $\approx$ C++

1.04× samples · 0.98× time

At $\varepsilon = 0.01$: Opposite Strengths

	BRAVA-GNN	KADABRA	
Kendall τ_b	0.87	0.39	9 of 9
top-100 overlap	58	87	9 of 9
runtime	0.6–52 s	1.4–19 s	

mean over 9 graphs

At $\varepsilon = 0.01$, Graph by Graph

At $\varepsilon = 0.0001$: KADABRA Wins Accuracy

	BRAVA-GNN	KADABRA	
Kendall τ_b	0.87	0.95	8 of 9
top-100 overlap	58	99.7	9 of 9
runtime	0.6–52 s	48 s–10.8 h	40–1365×

mean over 9 graphs · BRAVA-GNN unchanged

Answer to RQ2

BRAVA-GNN

fast

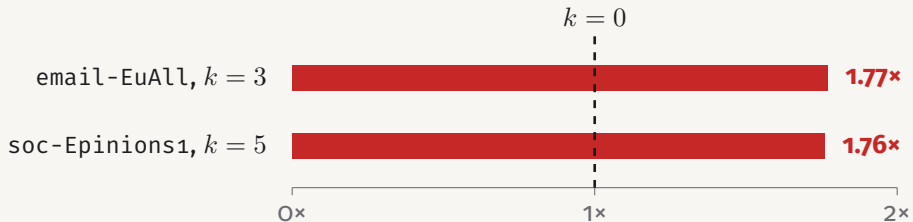
$0.6\text{--}52\text{ s} \cdot \tau_b$ 0.87

KADABRA

accurate

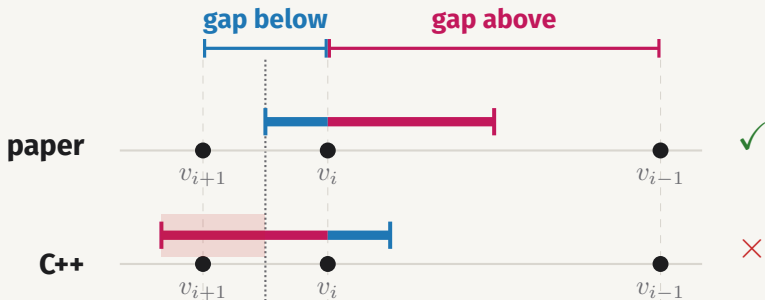
τ_b 0.95 · top-100 99.7

Asking for Less Costs More

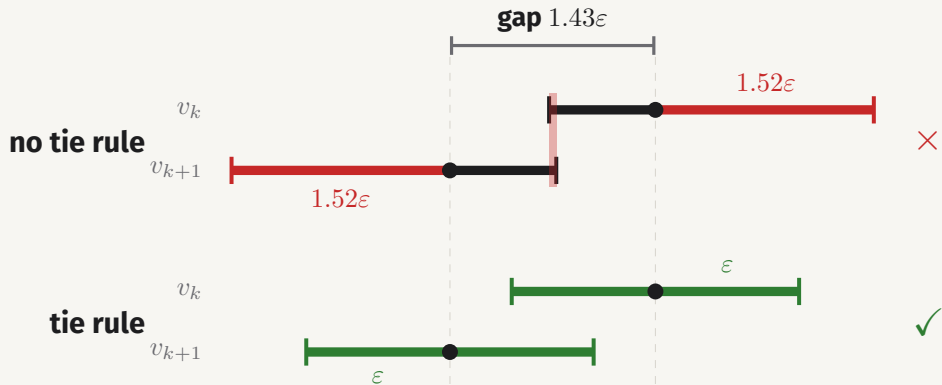


6 of 16 top- k configurations cost more than $k=0$

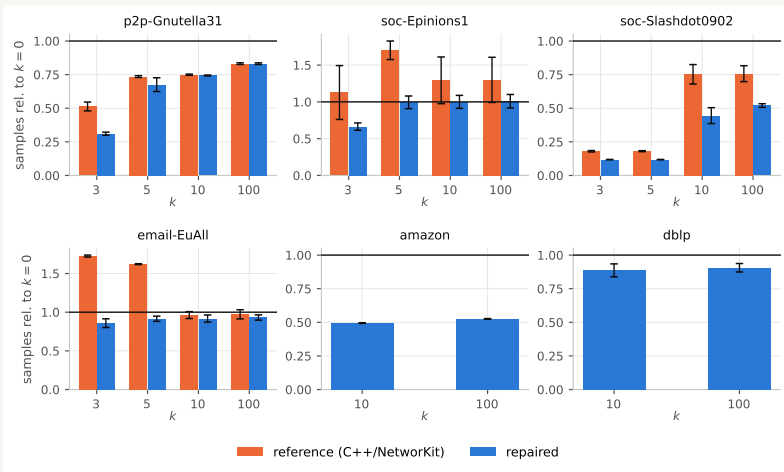
The Bug: Budgets Sized by the Wrong Gap



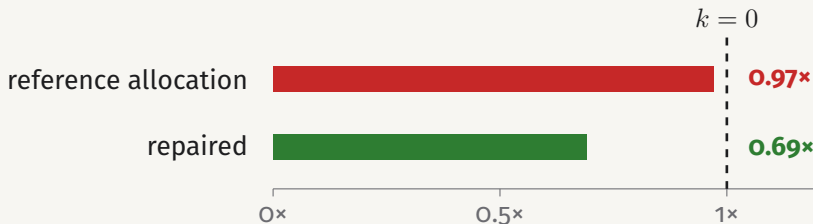
...and a Deadlock at the Top- k Boundary



Repaired: Top- k Never Costs More



Answer to RQ3



same top- k answers · fixed in `Graphs.jl` PR #518

Conclusion

RQ1 Julia $\approx$ C++

RQ2 speed or accuracy

RQ3 top- k saves work, once fixed

Limitations and Open Question

Who wins at a **fixed time budget**?

References

Algorithms

- [1] U. Brandes. A faster algorithm for betweenness centrality. *J. Math. Sociology* 25(2), 2001.
- [2] L. C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 1977.
- [3] I. Pohl. Bi-directional search. *Machine Intelligence* 6, 1971.
- [4] M. Borassi, E. Natale. KADABRA is an ADaptive Algorithm for Betweenness via Random Approximation. *ACM JEA* 24, 2019.
- [5] J. Dachille, A. Rossi, S. K. Maurya, F. Mallmann-Trenn, X. Liu, F. Giroire, T. Murata, E. Natale. Degree-Mass Message Passing for Betweenness Ranking in Directed and Undirected Networks. *CIKM '26*, 2026.

Data and software

- [6] J. Leskovec, A. Krevl. SNAP Datasets. snap.stanford.edu/data, 2014.
- [7] C. L. Staudt, A. Sazonovs, H. Meyerhenke. NetworkKit: A tool suite for large-scale complex network analysis. *Network Science* 4(4), 2016.
- [8] Graphs.jl, github.com/JuliaGraphs/Graphs.jl; KADABRA pull request #518.

Related work [9]–[15]: backup slides.

Thank you

questions?

Backup: Head-to-Head at $\varepsilon = 0.01$

Graph	BRAVA (CPU)	BRAVA (GPU)	KADABRA	τ_b BRAVA	τ_b KADABRA	Ovlp BRAVA	Ovlp KADABRA
p2p-Gnutella31	0.58 s	0.29 s	1.44 s	0.918 ± 0.006	0.763 ± 0.003	40 ± 13	75 ± 6
soc-Epinions1	0.59 s	0.58 s	1.95 s	0.917 ± 0.004	0.596 ± 0.006	73 ± 3	92 ± 1
soc-Slashdot0902	0.74 s	0.57 s	1.73 s	0.886 ± 0.007	0.574 ± 0.008	90 ± 3	92 ± 2
email-EuAll	1.19 s	0.66 s	12.92 s	0.991 ± 0.000	0.346 ± 0.006	69 ± 2	90 ± 2
com-youtube	5.89 s	3.17 s	3.81 s	0.902 ± 0.015	0.350 ± 0.006	59 ± 1	94 ± 1
amazon	14.06 s	9.14 s	6.19 s	0.835 ± 0.013	0.225 ± 0.004	54 ± 7	91 ± 2
dblp	24.32 s	14.75 s	11.49 s	0.880 ± 0.005	0.212 ± 0.004	54 ± 5	84 ± 2
cit-Patents	44.66 s	31.81 s	19.35 s	0.747 ± 0.005	0.212 ± 0.000	32 ± 3	71 ± 5
com-lj	52.27 s	38.40 s	17.89 s	0.782 ± 0.014	0.194 ± 0.003	51 ± 3	91 ± 4

$\varepsilon = 0.01, \delta = 0.1, k = 0, t = 8$; mean $\pm$ sd over 3 repetitions.

Backup: KADABRA at $\varepsilon = 0.0001$

Graph	τ_b	Ovlp	samples / ω	Time	vs. BRAVA (CPU)
p2p-Gnutella31	0.995	99	0.17	95.1 s	163×
soc-Epinions1	0.969	100	0.22	88.1 s	150×
soc-Slashdot0902	0.975	99	0.36	305.3 s	410×
email-EuAll	0.898	100	0.17	47.8 s	40×
com-youtube	0.955	100	0.73	8 042.5 s	1 365×
amazon	0.925	100	0.32	7 306.4 s	520×
dblp	0.962	100	0.14	9 394.8 s	386×
cit-Patents	0.945	99	0.13	20 904.2 s	468×
com-lj	0.918	100	0.40	38 735.2 s	741×

Backup: The Ground-Truth Bug

- The 8 test graphs from the `TestInstances` batch had corrupted ground truth: correct betweenness *values*, but paired with the *wrong node IDs*, as if the labels had been shuffled against the edge list.
- **Detection:** Kendall τ_b between degree sequence and ground-truth betweenness is reliably 0.7–0.9 for real networks, but ≈ 0 for the affected files.
- **Confirmation** on `p2p-Gnutella31`: the file's maximum value is numerically identical to the correct betweenness of a degree-95 hub, but attributed to a degree-1 leaf.
- **Resolution:** corrected files from the authors for the 5 ABCDE-sourced graphs; cached exact values for `p2p-Gnutella31` and `soc-Epinions1`; `soc-Slashdot0902` and `email-EuAll` recomputed by us; 5 excluded (four corrupted, plus `soc-LiveJournal1` without a file).

Graph	n	Reported Brandes time	Our measured time	Slowdown
soc-Slashdot0902	82 168	97.7 s	80 min	$\sim 49\times$
email-EuAll	265 214	96.4 s	169 min	$\sim 105\times$

Their dual-socket 52-core machine vs. our single-threaded `Graphs.jl` on a 24-core machine; extrapolated by $n \cdot m$, the other graphs would take weeks.

Backup: The Sample Budget ω and Burn-in N_0

- **Phase 1 (burn-in):** draw $N_0 = \lfloor \omega / \text{start_factor} \rfloor$ pairs (default 100) with *no* stopping check.
- **Phase 2:** keep sampling, checking the stopping condition periodically, until it holds or the ceiling ω is reached.

$$\omega = \frac{1}{2\varepsilon^2} \left(\log_2(D-1) + 1 + \ln \frac{0.5}{\delta} \right)$$

- The paper states $\frac{c}{\lambda^2} (\log_2(VD-2) + 1 + \ln \frac{2}{\delta})$; the reference C++ code instantiates $c = 1/2$ and uses $\ln \frac{0.5}{\delta}$. Our port reproduces the *code's* convention so the two stay comparable.
- ω alone bounds the error by ε with probability $1 - \delta/2$, so the per-vertex budgets may only sum to the remaining $\delta/2$.
- **Measured** at $\varepsilon = 0.0001$: samples/ ω is 0.14–0.36 on the six graphs of RQ1 and 0.13–0.73 over all nine — always well below the ceiling, but not predictable from size (com-youtube 0.73, cit-Patents 0.13).

Backup: KADABRA Main Loop Pseudocode

Algorithm 1: KADABRA Algorithm

Input: Graph G , error λ , failure probability δ

Output: Approximation $\tilde{bc}(v)$ such that $\Pr(\forall v, |\tilde{bc}(v) - bc(v)| \leq \lambda) \geq 1 - \delta$

$\omega \leftarrow \frac{c}{\lambda^2} (\lfloor \log_2(VD - 2) \rfloor + 1 + \log(\frac{2}{\delta}))$ // Max samples

$(\delta_L, \delta_U) \leftarrow \text{computeDelta}()$ // Heuristic error distribution

$N \leftarrow 0$

while $N < \omega$ **and not** $\text{haveToStop}(\tilde{bc}, \delta_L, \delta_U, \omega, N)$ **do**

$\pi \leftarrow \text{samplePath}()$ // Balanced Bidirectional BFS

foreach $v \in \pi$ **do**

$\tilde{bc}(v) \leftarrow \tilde{bc}(v) + 1$

$N \leftarrow N + 1$

return $\tilde{bc}(v)/N$ for all $v \in V$

Backup: Targets λ_L, λ_U

Burn-in estimates, sorted $v_1 \geq v_2 \geq \dots$ (compute_bet_err!)

$k = 0$: $\lambda_L(v) = \lambda_U(v) = \varepsilon$ for every vertex

Top- k , inside ($i \leq k$), paper:

$$\lambda_L(v_i) = \max\left(\varepsilon, \frac{\tilde{bc}(v_i) - \tilde{bc}(v_{i+1})}{2}\right) \quad \text{gap below}$$

$$\lambda_U(v_i) = \max\left(\varepsilon, \frac{\tilde{bc}(v_{i-1}) - \tilde{bc}(v_i)}{2}\right) \quad \text{gap above, } \lambda_U(v_1) = \infty$$

Top- k , outside ($i > k$): $\lambda_L(v_i) = \infty$, $\lambda_U(v_i) = \max(\varepsilon, \tilde{bc}(v_k) - \lambda_L(v_k) - \tilde{bc}(v_i))$

Tie rule: adjacent gap $< 2.5 \varepsilon \Rightarrow$ all four budgets of that pair $= \varepsilon$.

C++ reference: gap below and gap above swapped; adds $\lambda_L(v_k)$ instead of subtracting; tie rule never applied to (v_k, v_{k+1}) .

Backup: Split of δ and the Bounds f, g

Split (`compute_delta_guess!`), once, before the burn-in is discarded:

$$\delta_L(v) = e^{-C \lambda_L(v)^2 / \tilde{b}c(v)} + \frac{0.001 \delta}{4n}, \quad \delta_U(v) \text{ the same with } \lambda_U$$

C by bisection so that $\sum_v \delta_L(v) + \delta_U(v) \approx \delta/2$. Larger $\lambda \Rightarrow$ smaller δ ; $\lambda = \infty \Rightarrow \delta$ at its floor.

Bounds after N samples, with δ fixed (`compute_f`, `compute_g`):

$$f = \frac{\ln(1/\delta_L)}{N} \left(-a + \sqrt{a^2 + \frac{2 \tilde{b}c \omega}{\ln(1/\delta_L)}} \right), \quad a = \frac{\omega}{N} - \frac{1}{3}, \quad \text{at most } \tilde{b}c$$

$$g = \frac{\ln(1/\delta_U)}{N} \left(b + \sqrt{b^2 + \frac{2 \tilde{b}c \omega}{\ln(1/\delta_U)}} \right), \quad b = \frac{\omega}{N} + \frac{1}{3}, \quad \text{at most } 1 - \tilde{b}c$$

Stop test: $f, g \leq \varepsilon$, or separation in top- k — λ is never used here.

Backup: Top- k Allocation, Step by Step

Graph	k	Samples relative to $k = 0$		
		reference	+ gap pairing	+ boundary pair
email-EuAll	3	1.72 ± 0.01	1.24 ± 0.10	0.86 ± 0.06
	5	1.62 ± 0.01	1.65 ± 0.01	0.92 ± 0.03
	10	0.97 ± 0.05	0.92 ± 0.04	0.92 ± 0.05
	100	0.97 ± 0.05	0.93 ± 0.03	0.93 ± 0.03
p2p-Gnutella31	3	0.51 ± 0.04	0.31 ± 0.02	0.31 ± 0.01
	5	0.74 ± 0.01	0.68 ± 0.06	0.68 ± 0.05
	10	0.75 ± 0.00	0.74 ± 0.00	0.74 ± 0.00
	100	0.83 ± 0.01	0.83 ± 0.01	0.83 ± 0.01
soc-Epinions1	3	1.12 ± 0.37	0.67 ± 0.05	0.66 ± 0.05
	5	1.77 ± 0.01	1.42 ± 0.16	0.99 ± 0.09
	10	1.28 ± 0.31	1.00 ± 0.09	1.00 ± 0.09
	100	1.30 ± 0.31	1.01 ± 0.09	1.01 ± 0.09
soc-Slashdot0902	3	0.18 ± 0.00	0.12 ± 0.00	0.12 ± 0.00
	5	0.18 ± 0.01	0.12 ± 0.00	0.12 ± 0.00
	10	0.76 ± 0.07	0.44 ± 0.05	0.45 ± 0.06
	100	0.75 ± 0.07	0.52 ± 0.01	0.52 ± 0.01

our Julia port, samples relative to the same seed's $k = 0$; $\varepsilon = 10^{-4}$, $\delta = 0.1$, $t = 8$, three seeds

Backup: C++ vs. Julia Accuracy

Graph	Kendall τ_b		Top-100 overlap		Max. absolute error / ϵ		
	C++	Julia	C++	Julia	C++ as output	C++ rescaled	Julia, fixed
p2p-Gnutella31	0.995	0.995	99	98–99	4.9	0.18	0.26
soc-Epinions1	0.967	0.969	100	100	7.8	0.23	0.24
soc-Slashdot0902	0.973	0.975	100	99	12.3	0.31	0.39
email-EuAll	0.893	0.898	100	100	4.7	0.20	0.21
amazon	0.923	0.925	100	100	11.6	0.20	0.20
dblp	0.960	0.962	100	100	3.5	0.15	0.22

$\epsilon = 0.0001$, $\delta = 0.1$, $k = 0$, $t = 8$; τ_b mean and overlap range over two C++ and four Julia runs; max. error in units of ϵ .

Backup: Full Instance List

Instance	n	m	m/n	Reference
<i>Used in our experiments</i>				
p2p-Gnutella31	62 586	147 892	2.36	computed
soc-Epinions1	75 879	508 837	6.71	computed
soc-Slashdot0902	82 168	948 464	11.54	computed
email-EuAll	265 214	420 045	1.58	computed
com-youtube	1 134 890	2 987 624	2.63	provided
amazon	2 146 057	5 743 146	2.68	provided
cit-Patents	3 764 117	16 511 741	4.39	provided
com-lj	3 997 962	34 681 189	8.67	provided
dblp	4 000 148	8 649 011	2.16	provided
<i>Not used — no verified ground truth</i>				
web-Google	875 713	5 105 039	5.83	—
soc-Pokec	1 632 803	30 622 564	18.75	—
wiki-topcats	1 791 489	28 511 807	15.92	—
wiki-Talk	2 394 385	5 021 410	2.10	—
soc-LiveJournal1	4 847 571	68 993 773	14.23	—

Backup: Related Work

- **Brandes (2001)**: exact, $O(nm)$. Sub-quadratic exact ruled out under SETH.
- **RK** (Riondato & Kornaropoulos, 2014): VC-dimension bounds on the sample size.
- **ABRA** (Riondato & Upfal, 2018): tighter bounds via Rademacher averages; continued by SILVAN (2023) and Bavarian (2023).
- **KADABRA** (Borassi & Natale, 2019): martingale-based adaptive stopping + balanced bidirectional BFS; $> 100\times$ over prior approximation algorithms on $\approx 10k$ -node graphs.
- **GNN-Bet** (Maurya et al., 2019): learning-to-rank framing, but node-specific embeddings $\Rightarrow$ parameters scale with graph size ($\approx 58M$).
- **ABCDE** (Mirakyan, 2021): deep GCN with Progressive-DropEdge.
- **BRAVA-GNN** (Dachille et al., 2026): deterministic multi-hop degree-mass features, ≈ 1300 parameters, best τ_b on all 14 test graphs.

Backup: References, Related Work

- [9] M. Riondato, E. M. Kornaropoulos. Fast approximation of betweenness centrality through sampling. *WSDM '14*, 2014.
- [10] M. Riondato, E. Upfal. ABRA: Approximating betweenness centrality in static and dynamic graphs with Rademacher averages. *ACM TKDD* 12(5), 2018.
- [11] L. Pellegrina, F. Vandin. SILVAN: Estimating betweenness centralities with progressive sampling and non-uniform Rademacher bounds. *ACM TKDD* 18(3), 2023.
- [12] C. Cousins, C. Wohlgemuth, M. Riondato. Bavarian: Betweenness centrality approximation with variance-aware Rademacher averages. *ACM TKDD* 17(6), 2023.
- [13] S. K. Maurya, X. Liu, T. Murata. Fast approximations of betweenness centrality with graph neural networks. *CIKM '19*, 2019.
- [14] S. K. Maurya, X. Liu, T. Murata. Graph neural networks for fast node ranking approximation. *ACM TKDD* 15(5), 2021.
- [15] M. Mirakyan. ABCDE: Approximating betweenness-centrality ranking with progressive-DropEdge. *PeerJ Computer Science* 7, 2021.